Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Proving Basic Algorithms Correct: Sorting
Computer-Assisted Theorem Proving (7CCSACTP)
Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–5:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation (`simp`, `auto`, `nitpick`).
  - **Week 4:** Algebraic datatypes, termination measures, and tree induction.
  - **Week 5:** AI-assisted proving (Sledgehammer, RL loops, coding agents via MCP, DSP architecture).
- **Week 6 Milestone:** Verifying functional algorithm correctness and step bounds.
- Based on Tobias Nipkow's *Functional Data Structures* slides [Nipkow, 2020].

©Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–5:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation (`simp`, `auto`, `nitpick`).
  - **Week 4:** Algebraic datatypes, termination measures, and tree induction.
  - **Week 5:** AI-assisted proving (Sledgehammer, RL loops, coding agents via MCP, DSP architecture).
- **Week 6 Milestone:** Verifying functional algorithm correctness and step bounds.
- Based on Tobias Nipkow's *Functional Data Structures* slides [Nipkow, 2020].

- In Weeks 1 to 5, we progressed from foundations to modern AI automation:
- Week 1: ITP fundamentals and the Isabelle/HOL environment.
- Week 2: Mathematical induction and generalising induction hypotheses.
- Week 3: Functional lists, structural induction, and core tactics.
- Week 4: Custom algebraic datatypes, termination size measures, and tree induction.
- Week 5: AI-assisted theorem proving (Sledgehammer, RL loops, coding agents via MCP, DSP architecture).
- In Week 6, we study verified functional algorithms and data structures adapted from the Functional Data Structures lecture slides by Prof. Tobias Nipkow [Nipkow, 2020] at TU Munich (course website: `https://www21.in.tum.de/teaching/fds/`).

Outline

1. Correctness of sorting algorithms
   - Specification: `sorted` and multisets
   - Merge sort and computation induction
2. Running time
   - Counting function calls

Outline

1. Correctness of sorting algorithms
   - Specification: `sorted` and multisets
   - Merge sort and computation induction
2. Running time
   - Counting function calls

Correctness of Sorting Algorithms

```
sorted ::  ('a::linorder) list ⇒ bool

sorted [] = True
sorted (x # ys) =
  ((∀y ∈ set ys.  x ≤ y) ∧ sorted ys)
```


Correctness of Sorting Algorithms

sorted :: ('a::linorder) list ⇒ bool
sorted [] = True
sorted (x # ys) =
((∀y ∈ set ys. x ≤ y) ∧ sorted ys)

- We begin with the definition of sortedness on lists with linear order.
- The predicate sorted is defined recursively: the empty list and singleton list are trivially sorted.
- For a list with head x and tail ys, every element y in ys must be >= x, and ys itself must be sorted.
- So sorted means ascending, e.g. `sorted [1,2,2,3]`, but not `sorted [2,1]`.

Correctness of sorting

Specification of `sort :: ('a::linorder) list ⇒ 'a list`:

```
sorted (sort xs)
```

Is that it? How about

```
set (sort xs) = set xs
```

Better: every `x` occurs as often in `sort xs` as in `xs`.

More succinctly:

```
mset (sort xs) = mset xs
```

where `mset :: 'a list ⇒ 'a multiset`

Correctness of sorting
Specification of sort :: ('a::linorder) list ⇒ 'a list:
sorted (sort xs)
Is that it? How about
set (sort xs) = set xs
Better: every x occurs as often in sort xs as in xs.
More succinctly:
mset (sort xs) = mset xs
where mset :: 'a list ⇒ 'a multiset

- What constitutes the full specification of a sorting function?
- sorted(sort xs) alone is insufficient because a constant function returning [] satisfies it.
- set(sort xs) = set xs is also insufficient because sets ignore duplicates (e.g. [1, 1, 2] vs [1, 2]).
- The correct specification requires every element to occur with the exact same multiplicity in the output as in the input.
- Succinctly: mset (sort xs) = mset xs, where mset converts a list into a multiset.

What are multisets?

Sets with (possibly) repeated elements

Some operations:

```
      {#}  ::  'a multiset
add_mset   ::  'a ⇒ 'a multiset ⇒ 'a multiset
        +  ::  'a multiset ⇒ 'a multiset ⇒ 'a multiset
     mset  ::  'a list ⇒ 'a multiset
set_mset   ::  'a multiset ⇒ 'a set
```

Import `HOL-Library.Multiset`

What are multisets?
Sets with (possibly) repeated elements
Some operations:
{#} :: 'a multiset
add_mset :: 'a ⇒ 'a multiset ⇒ 'a multiset
+ :: 'a multiset ⇒ 'a multiset ⇒ 'a multiset
mset :: 'a list ⇒ 'a multiset
set_mset :: 'a multiset ⇒ 'a set
Import HOL-Library.Multiset

- Multisets are sets with possibly repeated elements (finite frequency mappings).
- Standard operations include {#} for empty multiset, add_mset for adding an element, and + for multiset sum.
- mset converts a list to a multiset; set_mset drops multiplicities to return the underlying set.
- All of these are provided by importing HOL-Library.Multiset.
- Examples: `add_mset 1 {#1,3#} = {#1,1,3#}`; `{#1,1#} + {#1,3#} = {#1,1,1,3#}`; `mset [1,3,1] = {#1,1,3#}`; `set_mset {#1,1,3#} = {1,3}`.